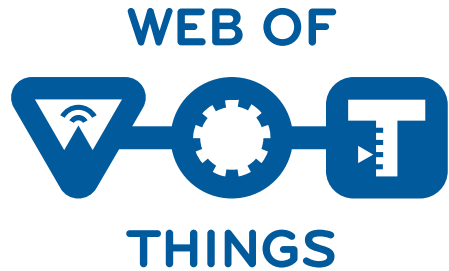


W3C Web of Things Community Group Online Meetup 33
(Joint Call with WebAgents CG)



Interoperable Cyber-Physical Multi-Agent Systems through Web of Things

Roman Binkert, TU Munich

Based on:

Binkert, R., Salama, F., Korkan, E., Käbis, S., Steinhorst, S. (2025). *Interoperable Cyber-Physical Multi-Agent Systems Through Web of Things*. In: Verma, H., Bozzon, A., Mauri, A., Yang, J. (eds) Web Engineering. ICWE 2025. Lecture Notes in Computer Science, vol 15749. Springer, Cham. https://doi.org/10.1007/978-3-031-97207-2_24

Roman Binkert

PhD Candidate

Embedded Systems and Internet of Things

Technical University of Munich

Research Interests

- Web of Things
- Interoperable IoT
- Industrial Metaverse

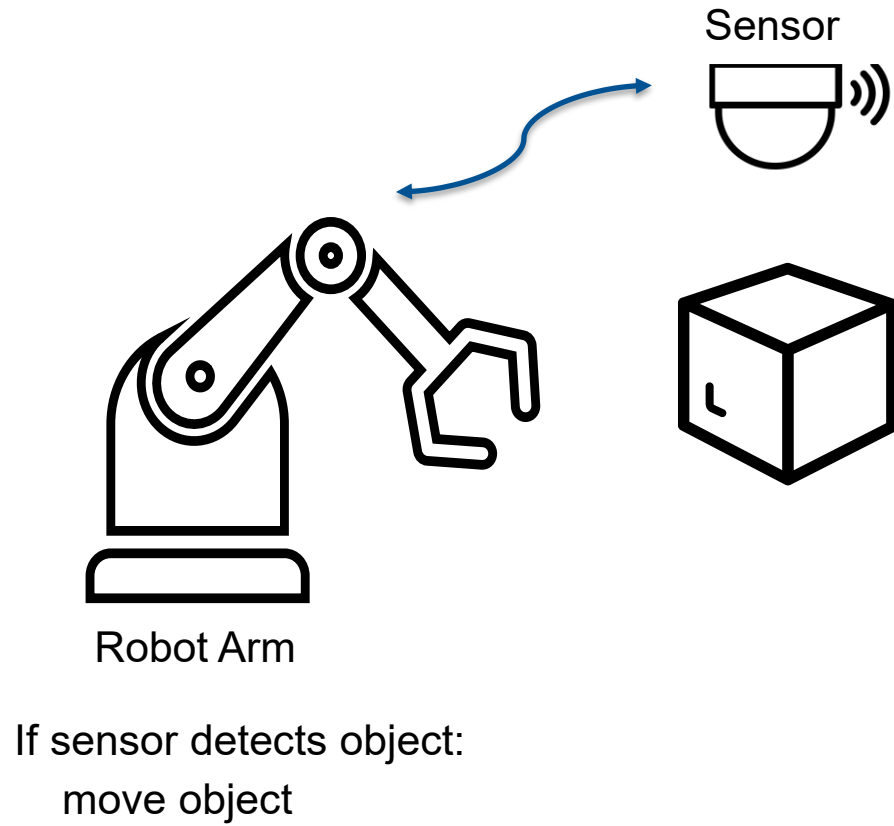
To Create:

Interoperable ...

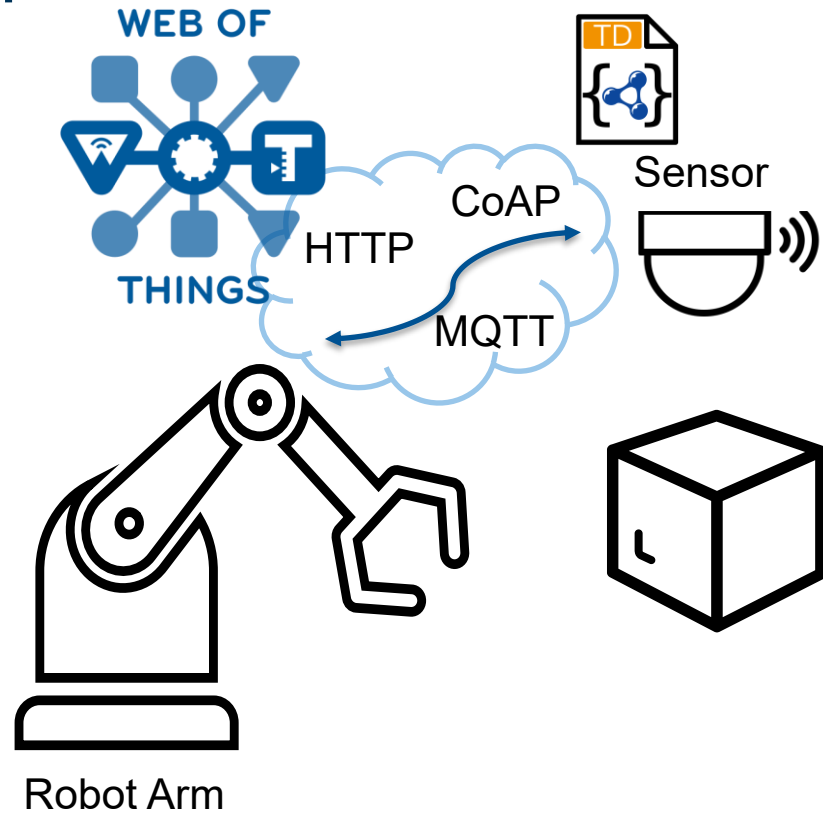
- Robotics
- Extended Reality (XR) Applications
- Cyber-Physical Agents



Motivation

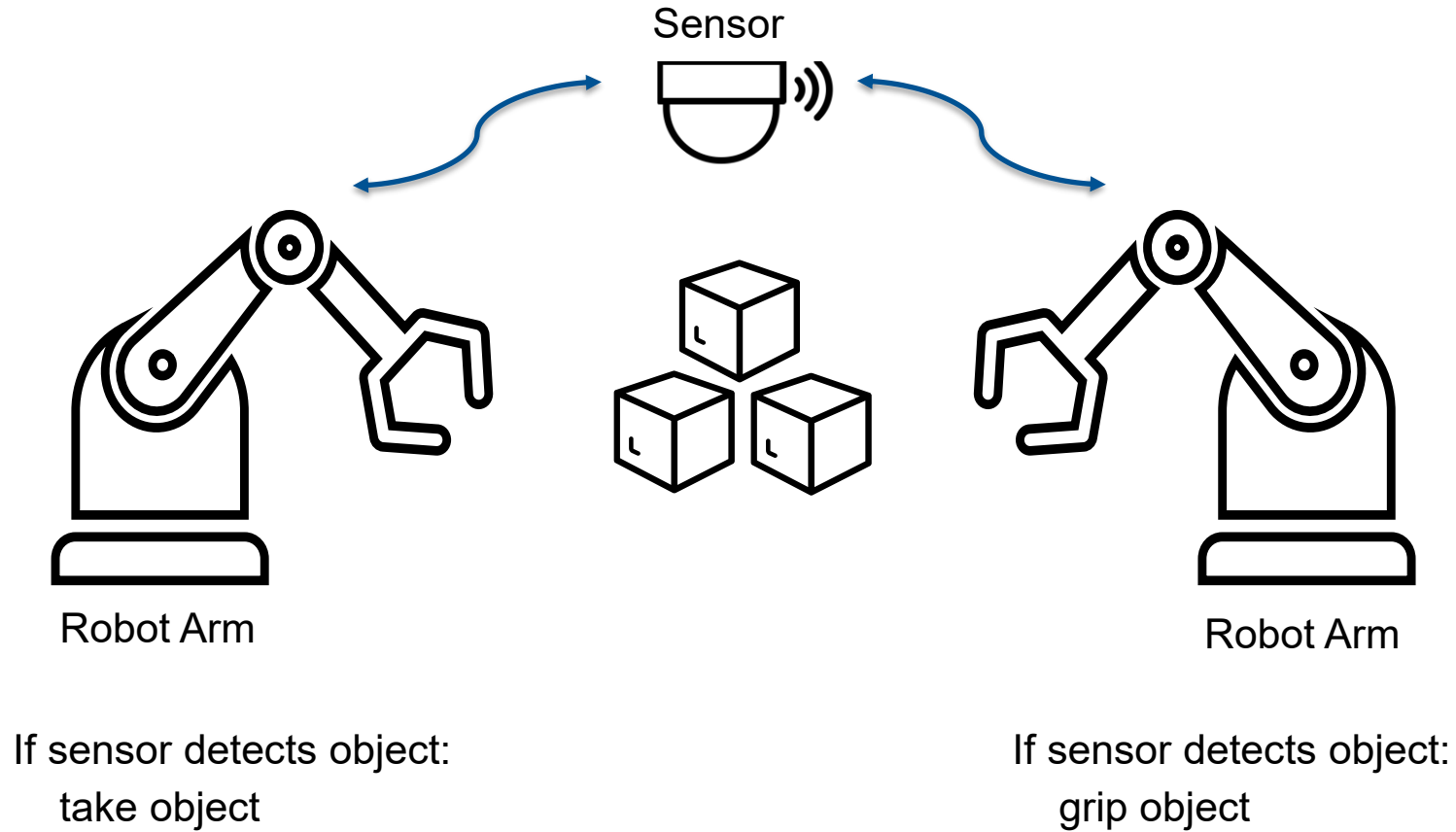


Motivation

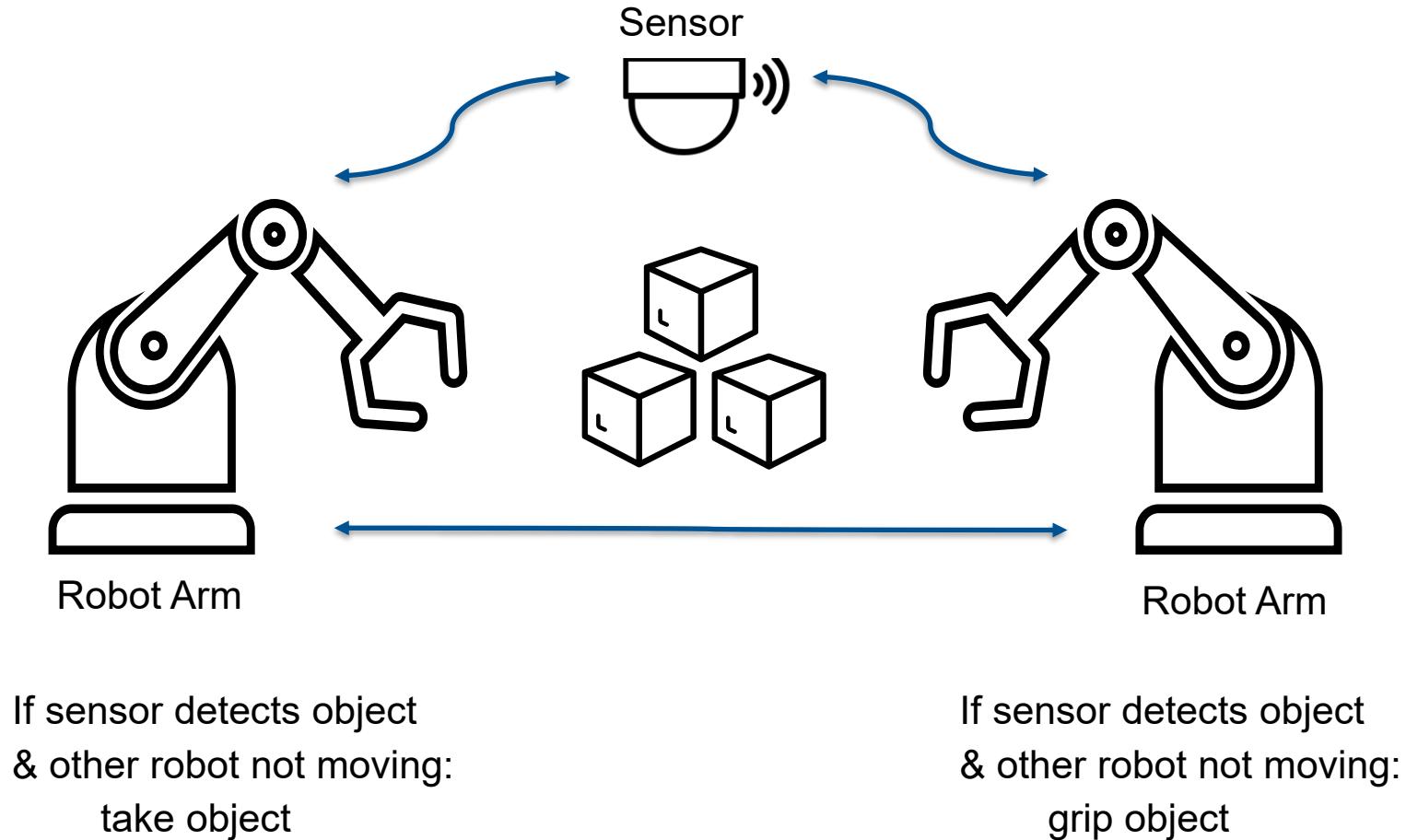


If sensor detects object:
move object

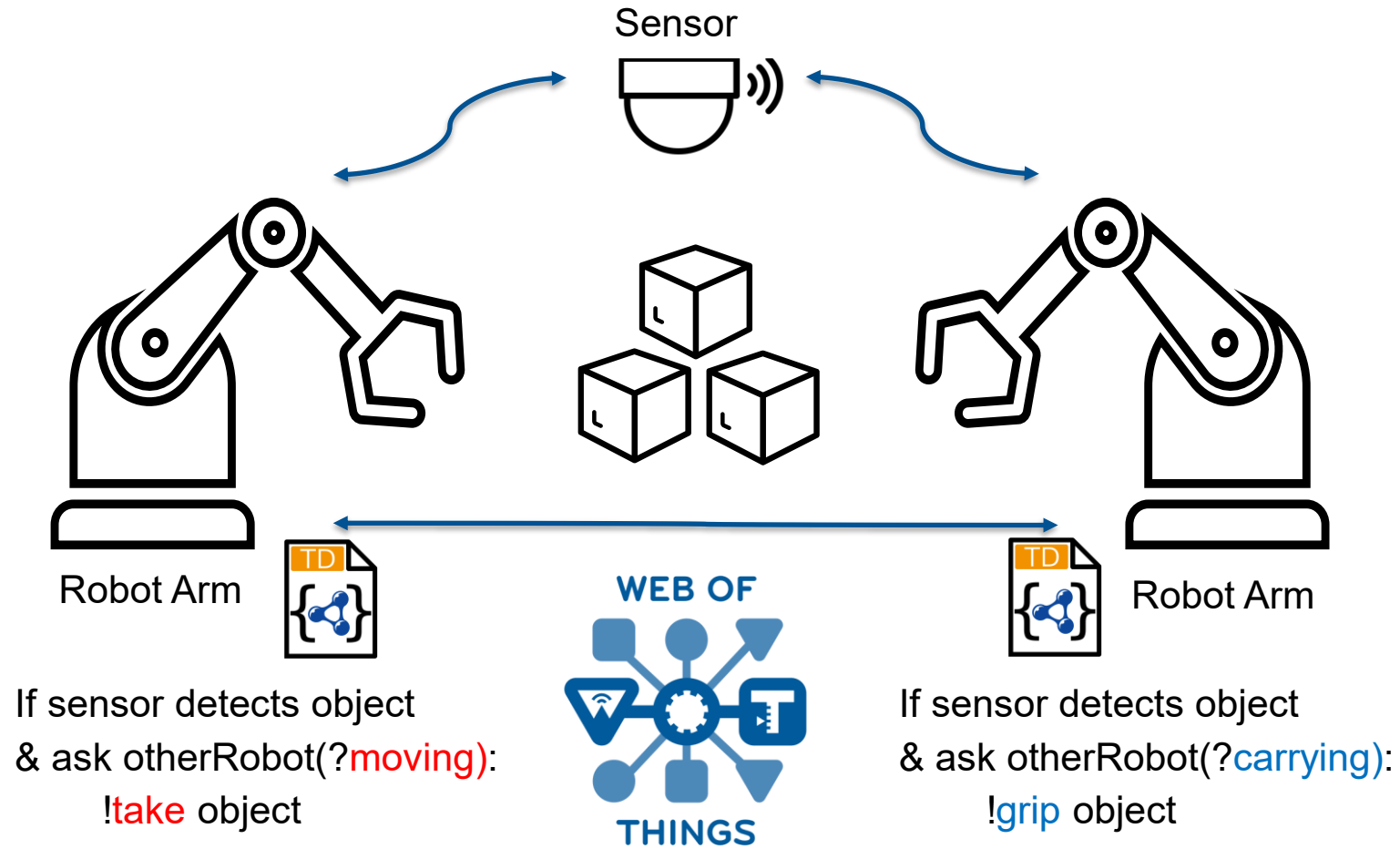
Motivation



Motivation



Motivation



Problem Statement

Multi-Agent Systems:

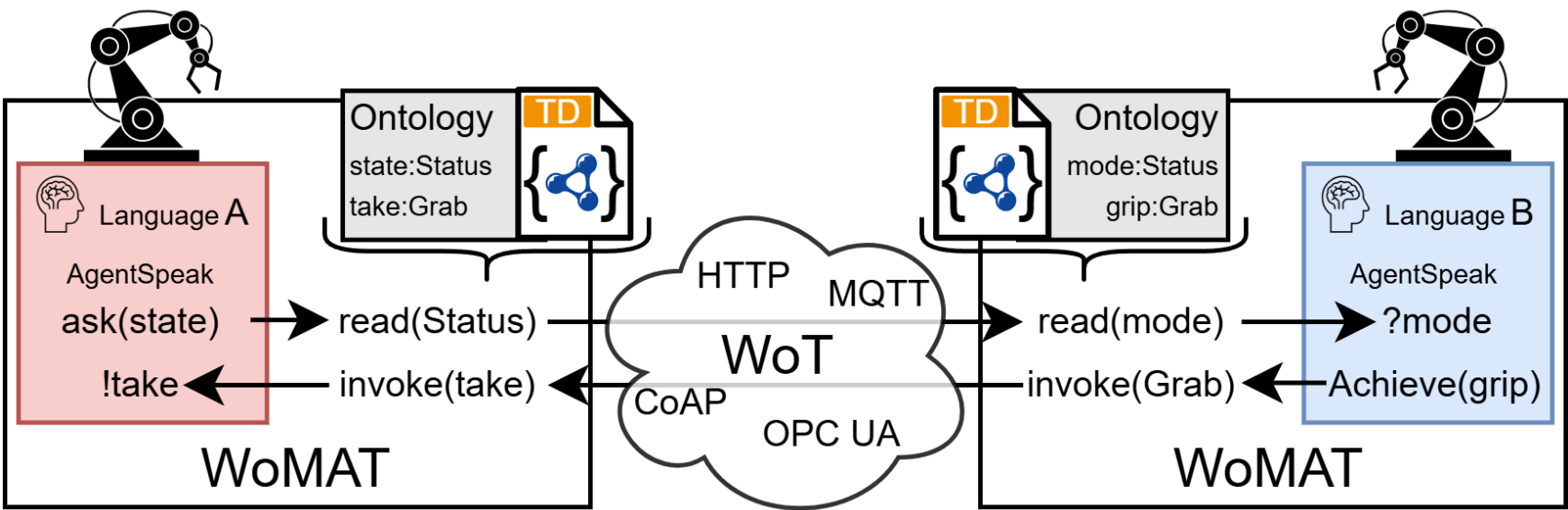
- good for autonomy and self-organization
 - goal-oriented programming and communication
- Lacks interoperability

Web of Things

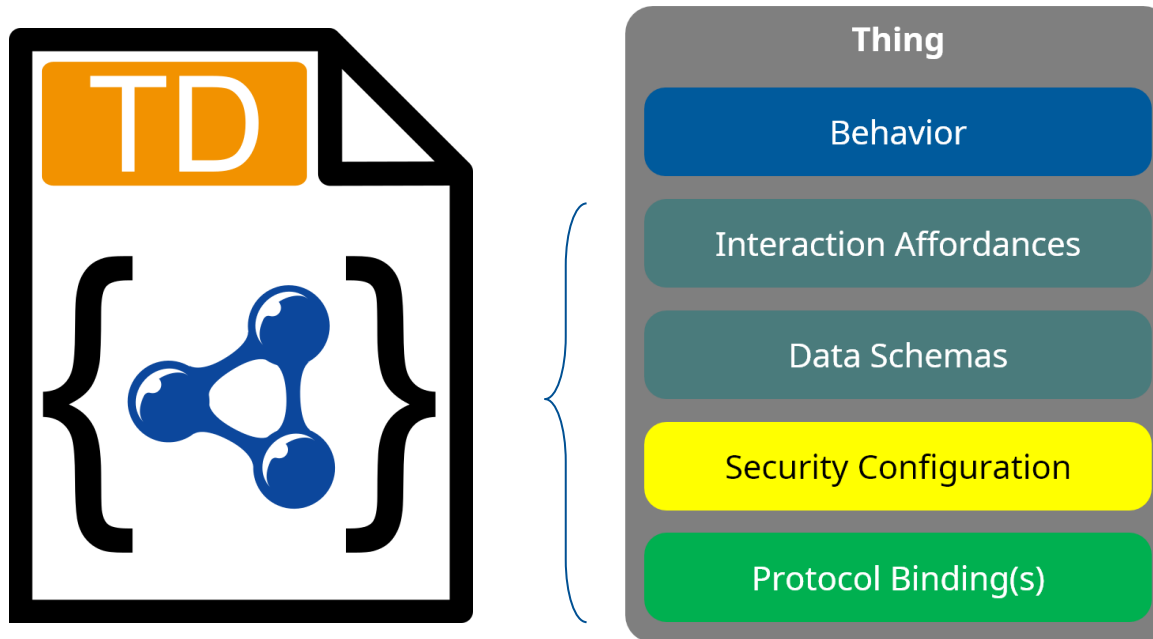
- Uniformly described capabilities of Things
- Protocol agnostic

Idea

Protocol & semantic agnostic
agent-to-agent and agent-to-thing
communication

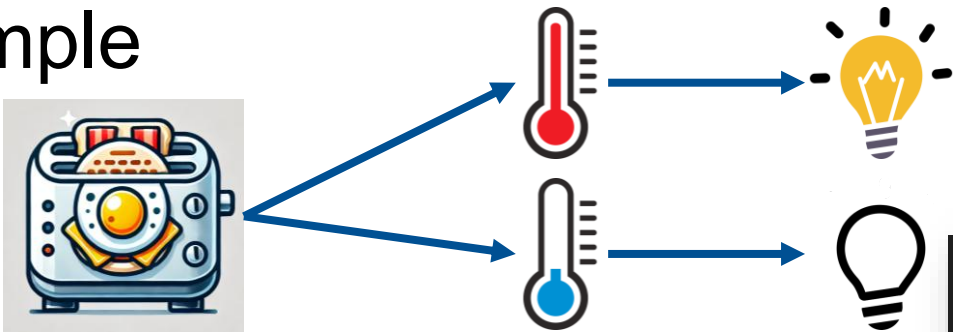


The Thing Description



W3C Standard describing the Web API provided by a Thing as part of the W3C Web of Things standard

Application Example



API

- Find API Description
- Find request

Request Params

- Adress
- FC: 0x03
- RA: 4001
- Scale
- RC: 2
- Endian

Data Params

- Data Format
- Unit

API

- Find API Description
- Find request

Request Params

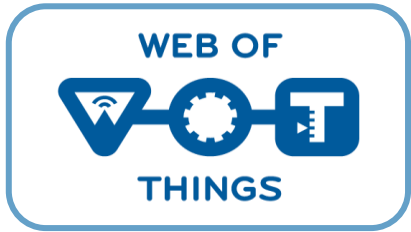
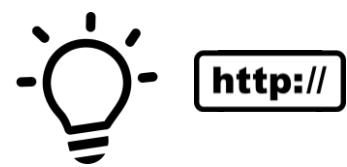
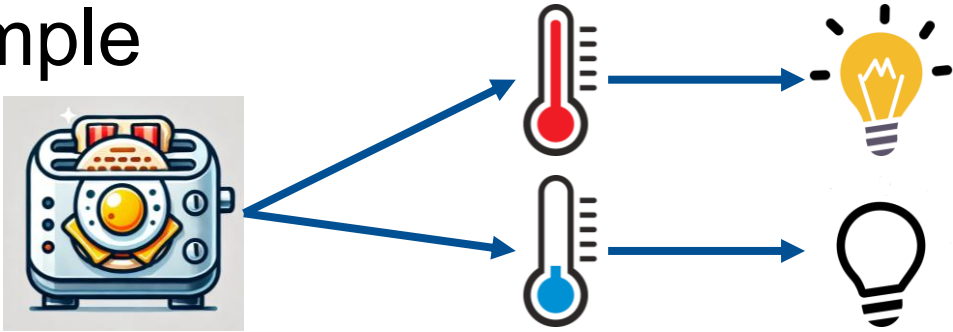
- Request Type
- Endpoint

Data Params

- Data Format
- Hue Type

```
1 # Modbus CONFIGURATION
2 DEVICE_IP = "192.168.1.100" # Replace with your sensor's IP
3 PORT = 502 # Default Modbus TCP port
4 DEVICE_ID = 1 # Modbus Slave ID
5 TEMP_REGISTER = 40001 # Register storing temperature
6 REGISTER_COUNT = 2 # Use 2 registers if the sensor stores a 32-bit float
7
8 # Initialize Modbus TCP Client
9 client = ModbusTcpClient(DEVICE_IP, port=PORT)
10 client.connect()
11
12
13 response = client.read_holding_registers(TEMP_REGISTER, REGISTER_COUNT, unit=DEVICE_ID)
14
15 if response.isError():
16     print("❌ Modbus error:", response)
17
18 # CASE 1: Sensor stores as 16-bit Integer (scaled by 10)
19 if REGISTER_COUNT == 1:
20     temperature = response.registers[0] / 10 # Assuming scaling factor of 10
21     print(f"🌡️ Temperature (Integer): {temperature}°C")
22
23 # CASE 2: Sensor stores as 32-bit Float (Two Registers)
24 elif REGISTER_COUNT == 2:
25     raw_data = struct.pack(">HH", response.registers[0], response.registers[1]) # Big-Endian
26     temperature = struct.unpack(">f", raw_data)[0] # Convert to float
27     print(f"🌡️ Temperature (Float): {temperature}°C")
28
29
30 # HTTP LAMP CONFIG
31 LAMP_API_URL = "http://192.168.1.150/" # Replace with your lamp's API endpoint
32 LAMP_API_HUE_ENDPOINT = LAMP_API_URL + "api/lamp/color" # Replace with your lamp's API endpoint
33
34 HTTP_METHOD = "POST" # or "PUT"
35 HEADERS = {"Content-Type": "application/json"}
36 payload = {
37     "hue": 60, # Yellow
38 }
39
40 if HTTP_METHOD.upper() == "PUT":
41     response = requests.put(LAMP_API_HUE_ENDPOINT, json=payload)
42 else:
43     response = requests.post(LAMP_API_HUE_ENDPOINT, json=payload)
```

Application Example



API

Request Params

Data Params

properties:
temperature:
unit: Celsius,
type: number,
forms:

API

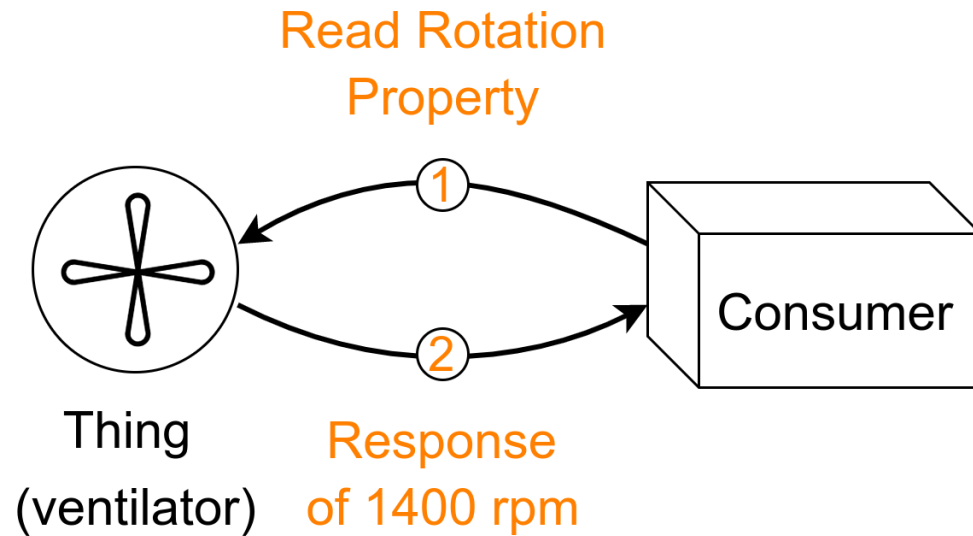
Request Params

Data Params

actions:
setColor:
unit: Hue,
type: number,
forms:

- Scan QR Codes
- readProperty(„temperature“)
- invokeAction(„setColor“, 60)

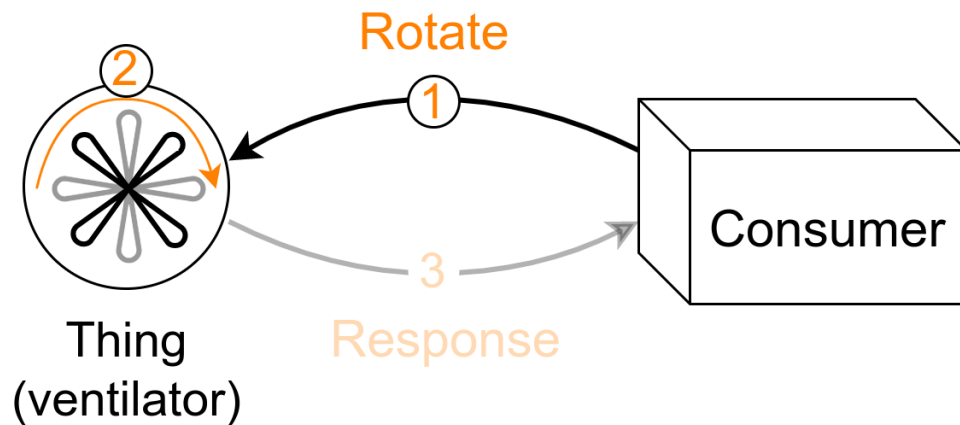
Interaction Affordances: Property



Reading, Writing a Property
("readproperty", "writeproperty")

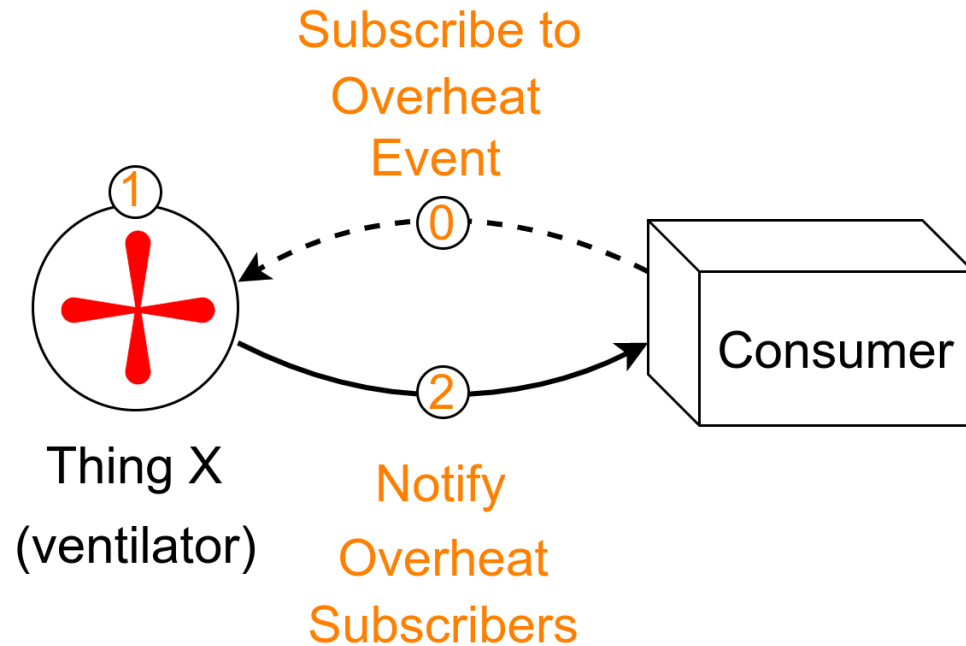
Observing, unobserving a Property
("observeproperty", "unobserveproperty")

Interaction Affordances: Action



Invoking an Action ("**invokeaction**")

Interaction Affordances: Event



Subscribing, Unsubscribing to/from an Event
("subscribeevent", "unsubscribeevent")

The Thing Description

```
1.  {
2.    "@context": ["https://www.w3.org/2022/wot/td/v1.1",
3.    {
4.      "saref": "https://saref.etsi.org/core/"
5.    }],
6.    "title": "SmartAC",
7.    "@type": "saref:Device",
8.    "securityDefinitions": {
9.      "basic_sc": {
10.        "scheme": "basic"
11.      }
12.    },
13.    "security": "basic_sc",
14.    "properties": {
15.      "isOn": {
16.        "type": "boolean",
17.        "@type": "saref:Property",
18.        "forms": [{
19.          "href": "https://smartac.example.com/ison"
20.        }]
21.      }
22.    },
23.    "actions": {
24.      "turnOn": {
25.        "title": "Turn On",
26.        "@type": "saref:OnCommand",
27.        "affects":
```

```
    {"id": "/properties/isOn"},
28.        "forms": [{
29.          "href": "https://smartac.com/turnon"
30.        }]
31.      },
32.      "turnOff": {
33.        "title": "Turn Off",
34.        "@type": "saref:offCommand",
35.        "affects": {"id": "/properties/isOn"},
36.        "forms": [{
37.          "href": "https://smartac.com/turnoff"
38.        }]
39.      }
40.    },
41.    "events": {
42.      "overheating": {
43.        "data": {
44.          "type": "string"
45.        },
46.        "forms": [{
47.          "href": "https://smartac.com/oh",
48.          "subprotocol": "longpoll"
49.        }]
50.      }
51.    }
52.  }
```

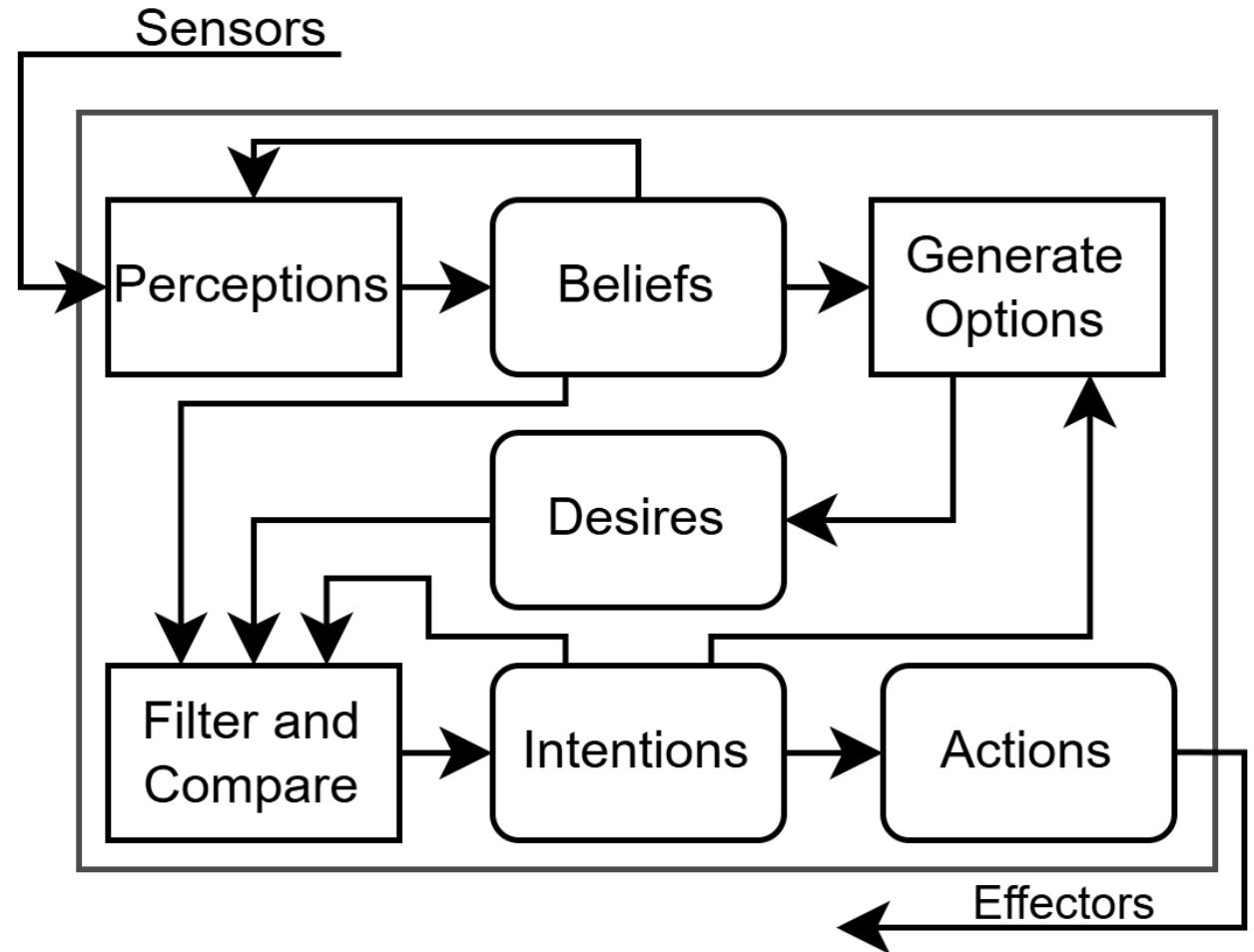

Agent Systems: Belief-Desire-Intention

Goal:

Program autonomous, proactive and reactive entities

Agent Plan Programming with Jason

```
+!process(green) <-  
  .print("Processing green");  
  !moveTo(0.2, - 0.35, 0.3);  
  !dropObject;  
  !goHome.
```

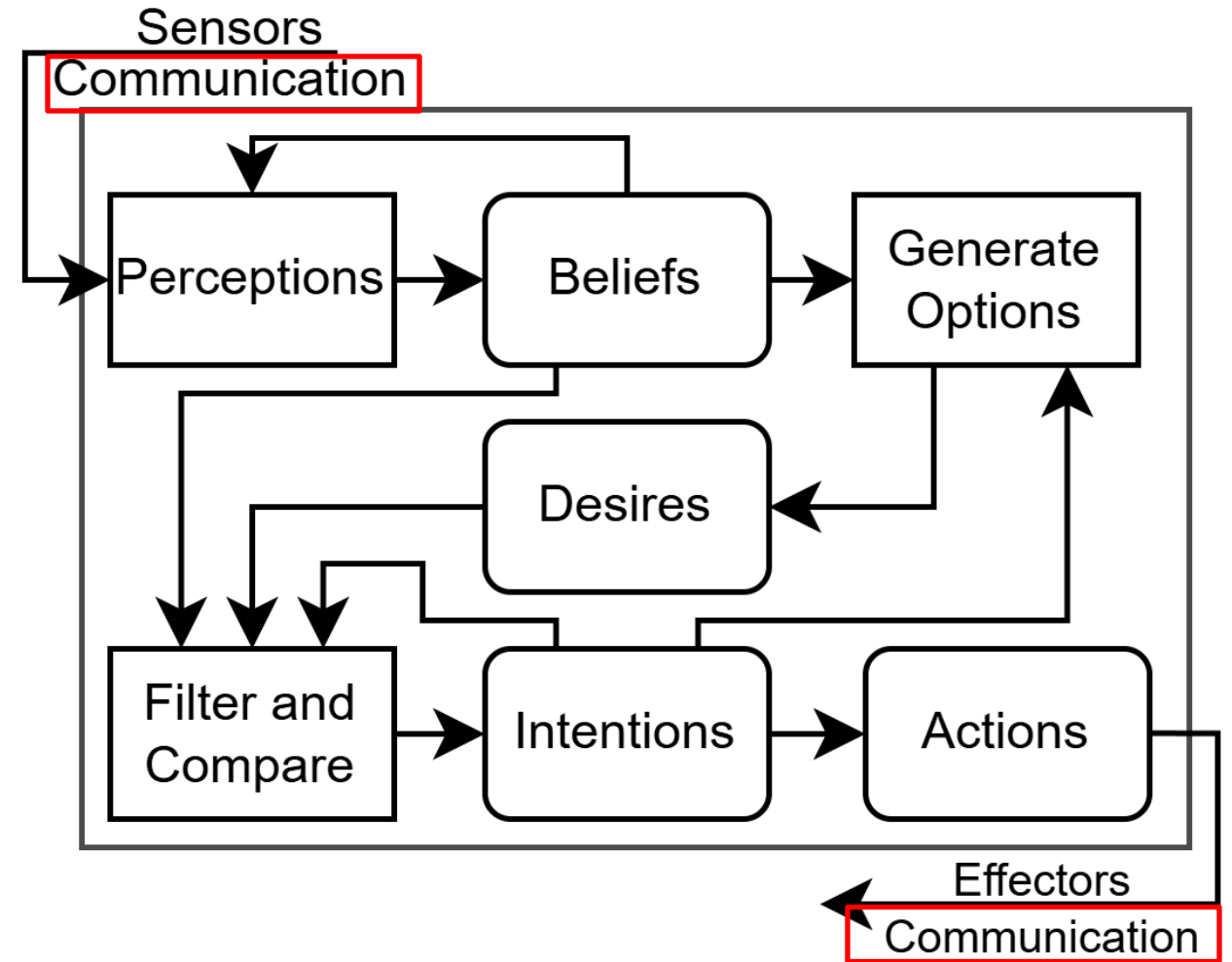


Multi-Agent Systems: Belief-Desire-Intention

Agent Communication

Performatives describe intention

- **Tell**: share belief
- **Ask**: does B know something
- **Achieve**: delegate goal
- **AskHow**: ask for plan how to reach goal

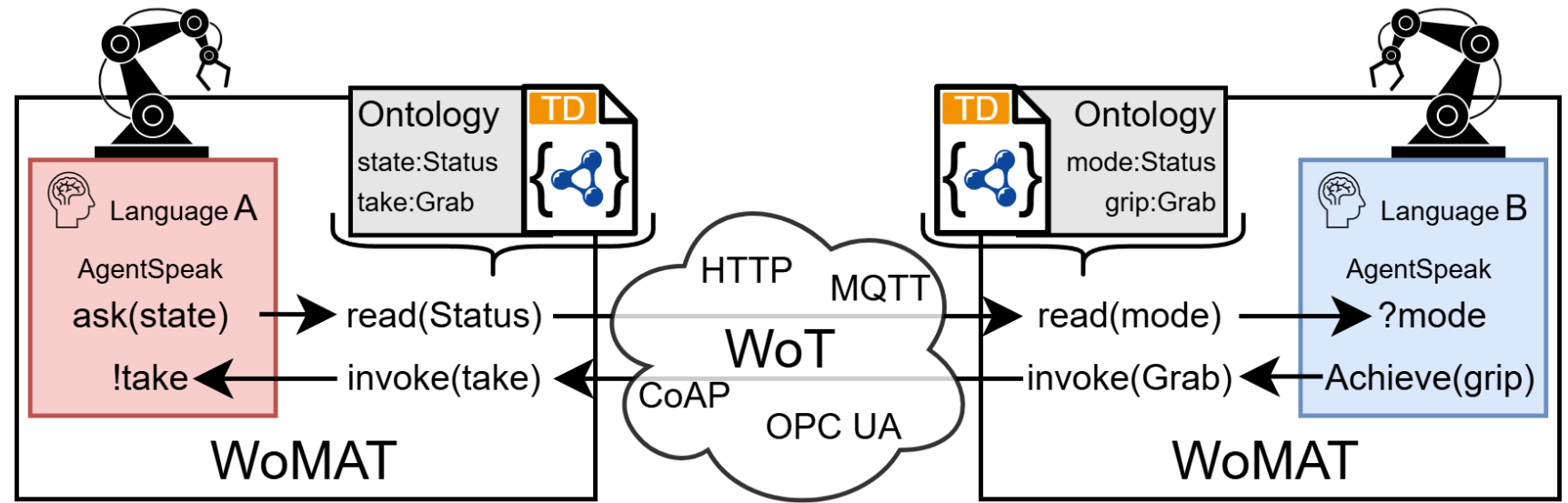


Idea Approach

Interoperable Cyber-Physical Multi-Agent Systems through Web of Things

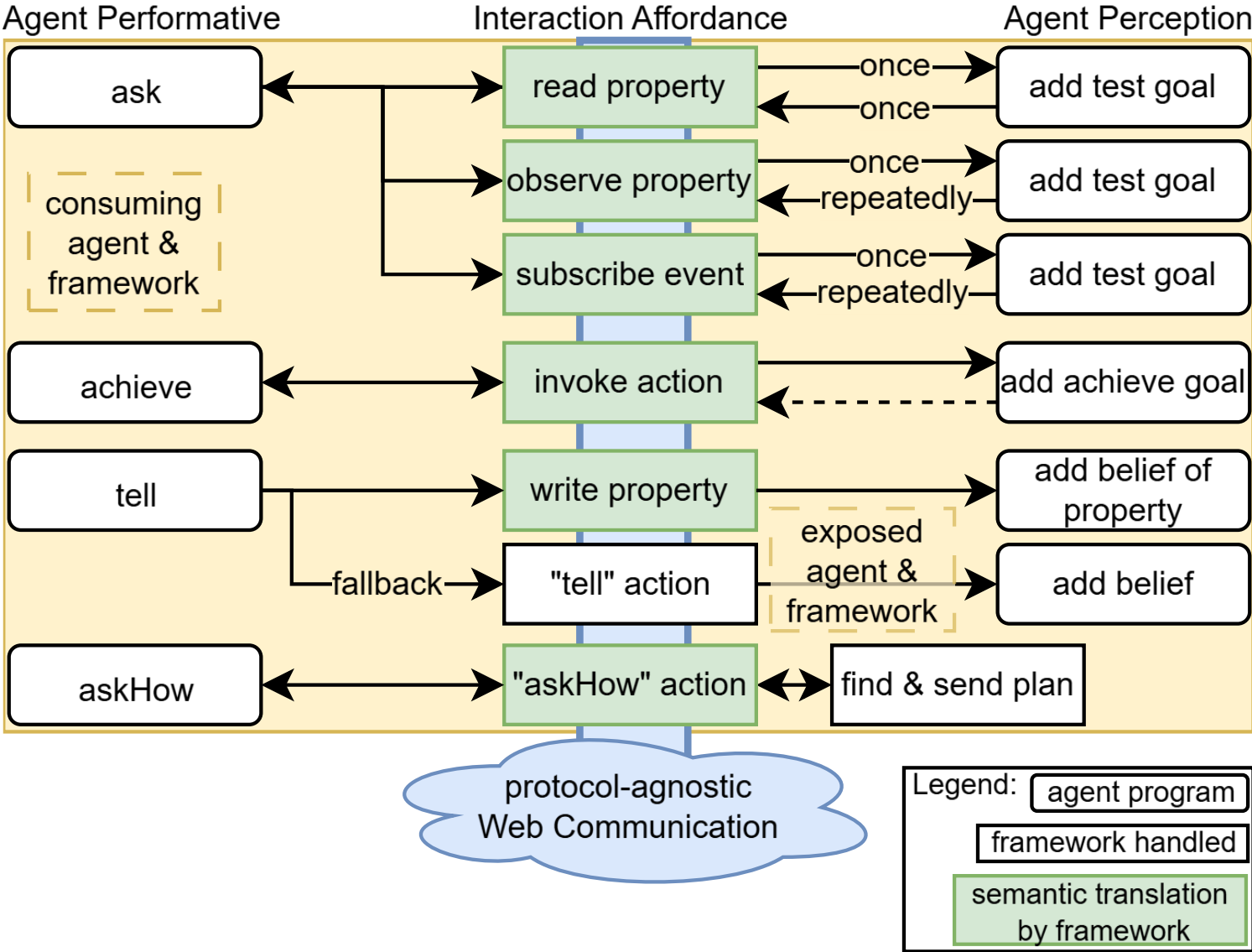
1. Match Agent Speak to WoT Interaction Affordances
2. Describe agents' capabilities with TD
3. Match internal naming to ontology in TD

→ Protocol & semantic agnostic agent-to-agent communication

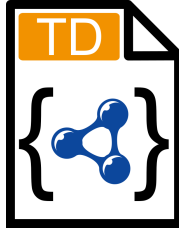


Match Agent Speak and WoT Interaction Affordances

Agents can interact with other agents and things equally



Semantic Translation

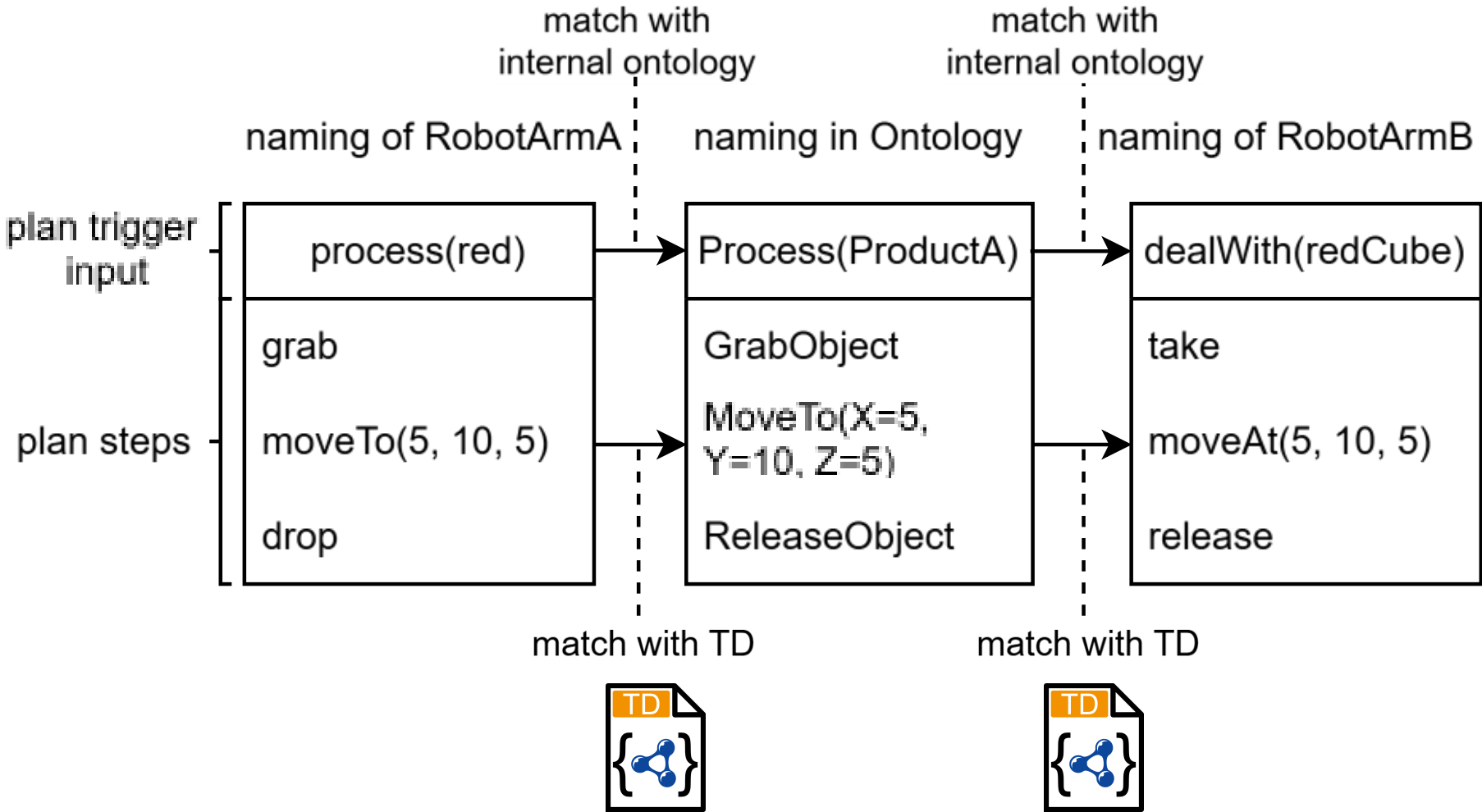


```
1 {"@type": "thing",
2  "title": "robot agent",
3  "@context": [{"eO": "exampleOntology"}],
4  "properties": { "state": { "@type": "eO:State" }},
5  "actions": {
6    "goHome": { "@type": "eO:GoHome"},
7    "moveTo": { "@type": "eO:MoveToPosition",
8      "input": { "properties": {
9        "x": { "@type": "eO:PositionX"},
10       "y": { "@type": "eO:PositionY"},
11       "z": { "@type": "eO:PositionZ" } } }},
12    "gripObject": { "@type": "eO:GrabObject" },
13    "dropObject": { "@type": "eO:ReleaseObject"},
14    "askHow": {},
15    "tell": {} }}
```

Agent Programming with Jason

```
+!process(green) <-
    .print("Processing green");
    !moveTo(0.2, - 0.35, 0.3);
    !dropObject;
    !goHome.
```

Semantic Plan Translation



Evaluation

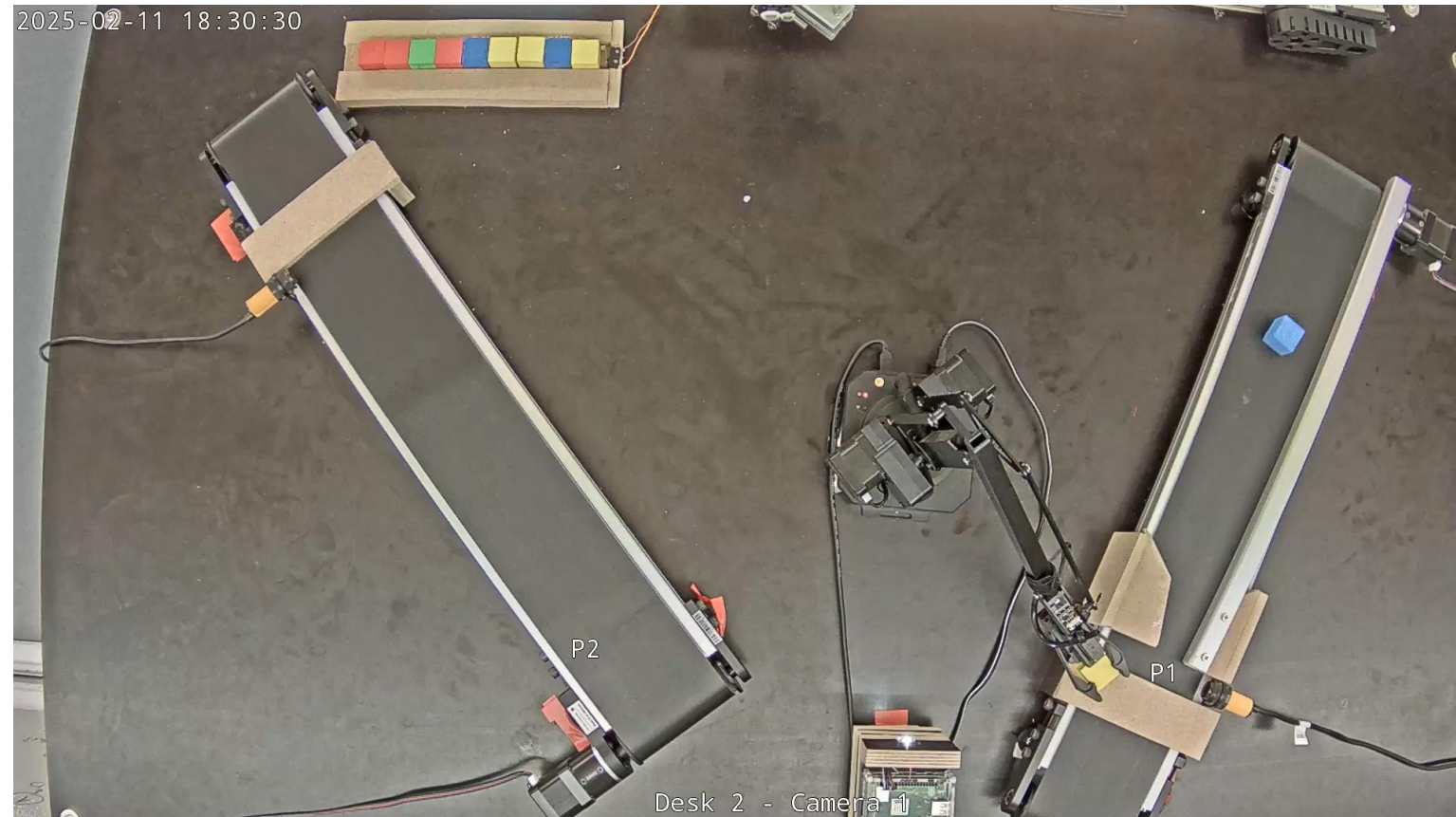
Agent to Thing Communication

Setup

- 2 robot arms
- 2 conveyor belts
- controlled by agents

Robot arm agents ask conveyor agents if there is a new object, then pick it up

Agents also control their „own“ device via WoT



Evaluation

Heterogenous Agent to Agent Communication and plan exchange

Setup

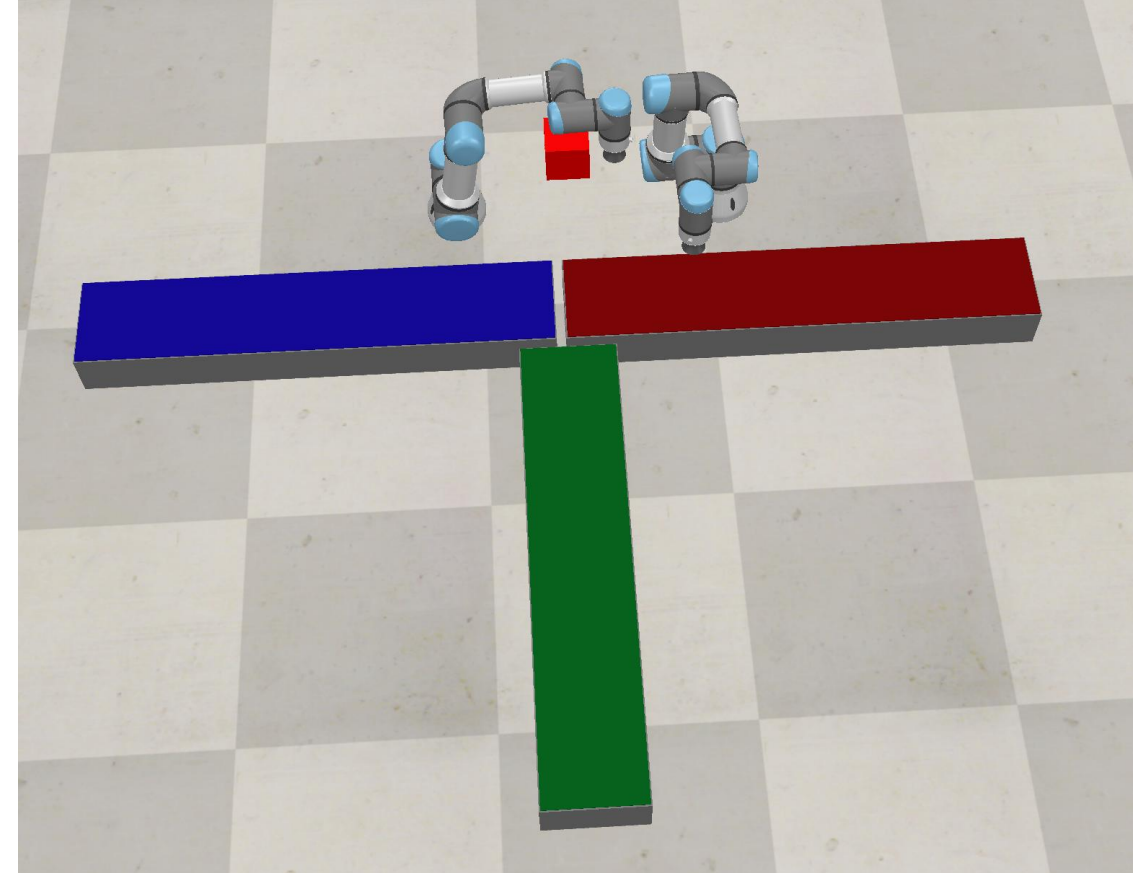
- 2 robot arms controlled by agents
- 3 Different products
- 3 conveyor belts (1 per product)

Goal: Move product on conveyor belt

Agent only knows plan for some of the products

→ The agent needs to ask the other agent for plan

Both agents use different internal naming



Evaluation

Heterogenous Agent to Agent Communication and plan exchange

Agent Plans

```
+objectDetected(Color) <-
```

```
  !pickup;  
  !process!(Color)
```

```
+!process(blue) <-
```

```
  // @WoT  
  !moveTo(0.05, -0.05, 0.4);  
  !dropObject;  
  !goHome.
```

```
+!process(red) <-
```

```
  // @WoT  
  !moveTo(0.3, -0.05, 0.4);  
  !dropObject;  
  !goHome.
```

```
"goHome": {"@type": "testOntology:GoHome",
```

```
"moveTo": {
```

```
  "@type": "testOntology:MoveToPosition",
```

```
  "input": {
```

```
    "type": "object",
```

```
    "properties": {
```

```
      "x": {
```

```
        "type": "number",
```

```
        "@type": "testOntology:PositionX",
```

```
        "y": {...},
```

```
        "z": {...}
```

```
+objectDetected(Color) <-
```

```
  !pickup;
```

```
  !dealWith!(Color)
```

```
+!dealWith(red) <-
```

```
  // @WoT
```

```
  !moveAt(0.3, -0.05, 0.4);
```

```
  !release;
```

```
  !homeing.
```

```
"homeing": {"@type": "testOntology:GoHome",
```

```
"moveAt": {
```

```
  "@type": "testOntology:MoveToPosition",
```

```
  "input": {
```

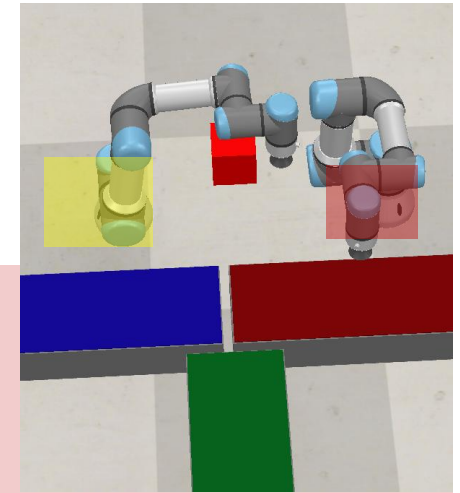
```
    "type": "object",
```

```
    "properties": {
```

```
      "X_Value": {
```

```
        "type": "number",
```

```
        "@type": "testOntology:PositionX"},
```



Evaluation

Heterogenous Agent to Agent and plan exchange

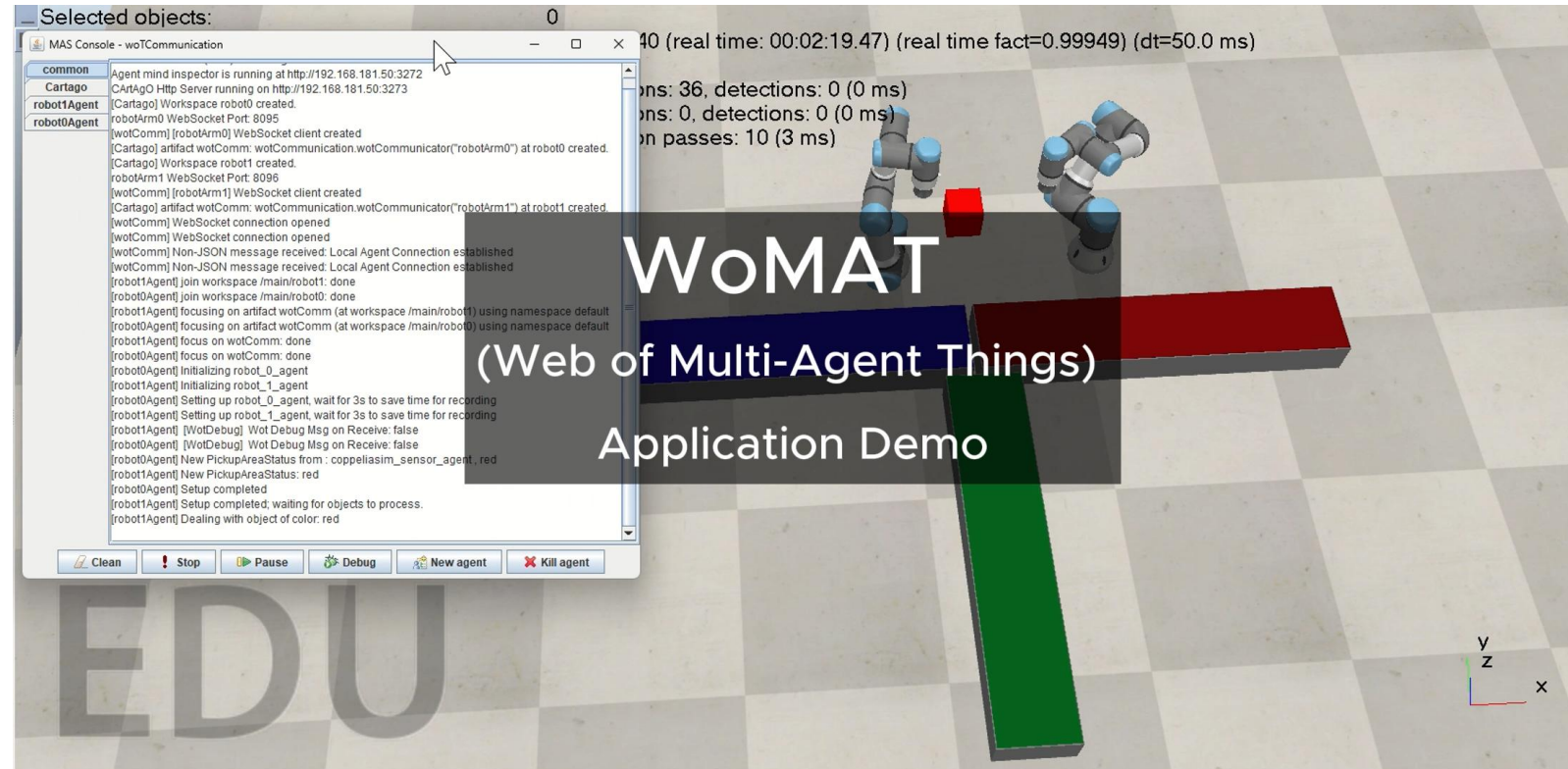
Setup

- 2 robot arms controlled by agents
- 3 conveyor belts
- Different products

Agent only knows plan for some of the products

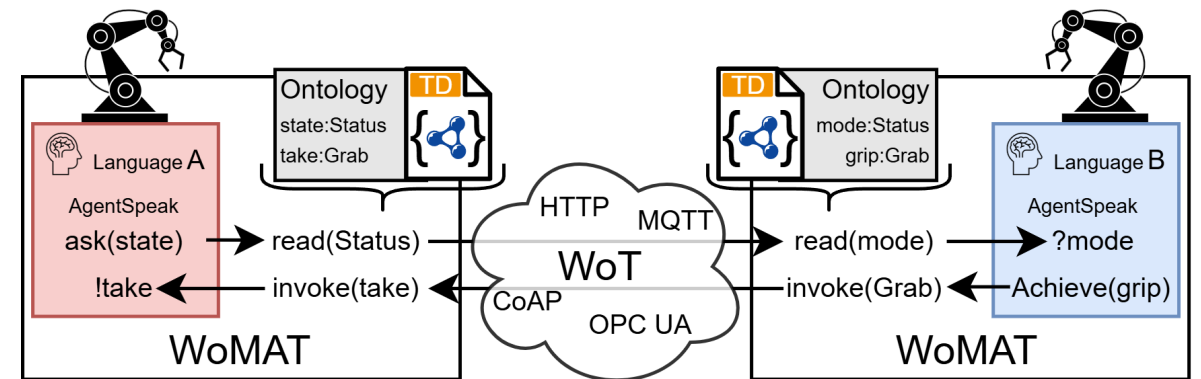
→ The agent needs to ask the other agent for plan

Both agents use different internal naming



Conclusion

- Protocol-agnostic agent-to-agent and agent-to-thing communication with agent speak
- Semantic translation between different concrete namings based on TD and ontology
- Evaluation:
 - Agent-controlled cyber-physical production line
 - Plan exchange between heterogeneous agents
 - Low impact on performance



Thank you for your attention!

References

1. FIPA ACL Message Structure Specification. http://www.fipa.org/specs/fipa00061/SC00061G.html#_Toc26669706. Accessed 02 Apr 20242.
2. FIPA Communicative Act Library Specification. http://www.fipa.org/specs/fipa00037/PC00037E.html#_Toc505673625. Accessed 02 Apr 20243.
3. Amrani, N.E.A., Snineh, S.M., Youssfi, M., Abra, O.E.K., Bouattane, O.: Interoperability model between heterogeneous MAS platforms based on mobile agent and reinforcement learning. In: ICDS (2021). <https://doi.org/10.1109/ICDS53782.2021.9626723>
4. Bienz, S., Ciorrea, A., Mayer, S., Gandon, F., Corby, O.: Escaping the streetlight effect: semantic hypermedia search enhances autonomous behavior in the web of things. In: IoT '19 (2019). <https://doi.org/10.1145/3365871.3365901>.
5. Boissier, O., Bordini, R.H., Hübner, J.F., Ricci, A.: Multi-agent oriented programming - the jacamo platform (2016). <https://www.emse.fr/~boissier/enseignement/maop16/pdf/aop.pdf>.
6. Bordini, R.H., Hübner, J.F.: BDI agent programming in agentspeak using jason(2006). https://doi.org/10.1007/11750734_9.
7. Charpenay, V., Zimmermann, A., Lefrançois, M., Boissier, O.: Hypermedea: a framework for web (of things) agents. In: WWW 2022 (2022). <https://doi.org/10.1145/3487553.3524243>
8. Ciorrea, A., Boissier, O., Ricci, A.: Engineering world-wide multi-agent systems with hypermedia. In: EMAS (2019). https://doi.org/10.1007/978-3-030-25693-7_15
9. Ciorrea, A., Mayer, S., Michahelles, F.: Repurposing manufacturing lines on the fly with multi-agent systems for the web of things. EMAS (2018)
10. Dorri, A., Kanhere, S., Jurdak, R.: Multi-agent systems: a survey. IEEE Access 6(2018). <https://doi.org/10.1109/ACCESS.2018.2831228>
11. Kampik, T., et al.: Governance of autonomous agents on the web: challenges and opportunities. ACM Trans. Internet Technol. (2022). <https://doi.org/10.1145/3507910>
12. Lizarin, N.M., Alexandre, T., Moreira de Paiva, M., Pantoja, C.E., Viterbo, J., Bernardini, F.: A decentralized agent-based model for crisis events using embedded systems. In: PAAMS (2025). https://doi.org/10.1007/978-3-031-70415-4_14
13. Matsukura, R., McCool, M., Toumura, K., Lagally, M.: Wot architecture 1.1. Recommendation, W3C (2023). <https://www.w3.org/TR/2023/REC-wot-architecture11-20231205/>
14. McCool, M., Korkan, E., Käbis, S.: Wot thing description 1.1. Recommendation, W3C (2023). <https://www.w3.org/TR/2023/REC-wot-thing-description11-20231205/>
15. Noura, M., Siegert, V., Gaedke, M.: Wat: autonomous hypermedia-driven web agents for web of things devices. In: CEUR Workshop Proceedings (2021). <https://ceur-ws.org/Vol-3111/short6.pdf>
16. Pantoja, C.E., Souza de Jesus, V., Lizarin, N.M., Viterbo, J.: a spin-off version of jason for iot and embedded multi-agent systems. In: Intelligent Systems (2023). https://doi.org/10.1007/978-3-031-45368-7_25
17. Poslad, S., Charlton, P.: Standardizing agent interoperability: the fipa approach(2001). https://doi.org/10.1007/3-540-47745-4_5

Thank you for your attention

Contact Details:

Roman Binkert
Research Associate

Associate Professorship of Embedded Systems and Internet of Things
School of Computation, Information and Technology
Technical University of Munich

E-Mail: roman.binkert@tum.de



Homepage



Email